

Programa · Comisión 2026

AI Engineering

Clase 4:

Escalabilidad documental: RAG avanzado y Pinecone

Objetivos

1. Configurar una infraestructura vectorial en la nube con Pinecone Serverless.
2. Diseñar estrategias de ingesta masiva con metadatos estructurados para filtrado.
3. Construir un sistema de recuperación híbrida combinando BM25 y búsqueda vectorial.
4. Evaluar el rendimiento de un sistema de recuperación con métricas de Precision y Recall.
5. Aplicar re-ranking y fusión de resultados (RRF) para maximizar la precisión final.

Agenda

1. Pinecone Serverless: persistencia vectorial en la nube.
2. Ingesta masiva y gestión de metadatos avanzados.
3. Métricas y recuperación híbrida: precisión, recall y re-ranking.
4. Pre-Entrega 4: tu sistema RAG escalable en la nube.

**¡Esta clase va a ser
grabada!**



01

Pinecone Serverless

¿Por qué Pinecone?

De local a la nube

ChromaDB es excelente para desarrollo local y prototipado rápido, pero en producción aparecen desafíos de escalabilidad y alta disponibilidad.

Pinecone es una base de datos vectorial nativa de la nube, diseñada para manejar millones de vectores con latencias de milisegundos.

Arquitectura Serverless: no gestionás nodos ni memoria; pagás por lo que usás y el almacenamiento escala solo.

Conceptos Clave de Infraestructura

Index, Namespace y Metadatos

- **Index:** el contenedor principal. Define la dimensión de los vectores y la métrica de similitud a usar.
- **Namespace:** segmenta un mismo índice en particiones lógicas, vital para multi-tenancy o separar dev/staging/prod.
- **Metadatos:** información JSON adjunta al vector que permite filtrar durante la búsqueda.

Separación de cómputo y almacenamiento: las búsquedas son rápidas sin importar cuántos datos tengas guardados.

Local vs. Serverless

Cuándo dar el salto a la nube

ChromaDB local: ideal para prototipado y desarrollo rápido, corriendo en tu propia máquina, sin alta disponibilidad garantizada.

Pinecone Serverless: corre en infraestructura administrada en la nube, escala automáticamente y separa cómputo de almacenamiento.

¿Cuál usar? Local para iterar rápido en desarrollo; Pinecone cuando el sistema necesita persistencia y escala de producción.

El Flujo de Trabajo

Ingesta y Recuperación

Ingesta: los documentos se fragmentan (chunking), se convierten a vectores (embedding) y se suben con upsert.

Recuperación: la consulta del usuario se vectoriza y se busca su vecino más cercano en Pinecone.

Filtrado: la búsqueda se acota filtrando por metadatos antes de comparar similitud.

Resultado: los fragmentos recuperados alimentan al LLM para generar la respuesta final.

Errores Comunes de Arquitectura

Mismatch de Dimensiones

Insertar un vector de 1536 dimensiones en un índice configurado para 768 lanza un error inmediato.

Ignorar los Namespaces

Mezclar datos de prueba con producción, o de distintos usuarios, compromete seguridad y claridad.

Métrica Errónea

Usar similitud euclidiana cuando el modelo de embeddings fue optimizado para similitud coseno.

Evolución Vectorial

De FAISS a la nube

FAISS: local, en memoria, sin persistencia ni escalabilidad. El punto de partida de casi todo motor vectorial.

ChromaDB y LanceDB suman persistencia en disco y buen rendimiento, pero siguen operando en una sola máquina.

Pinecone: servicio gestionado en la nube, con escalabilidad elástica y alta disponibilidad. El siguiente escalón hacia producción real.

Arquitectura Interna de Pinecone

SDK y Aplicación

Tu app se autentica y envía peticiones mediante el SDK oficial, que maneja la serialización.

Index, Namespaces y Shards

El índice se particiona lógicamente en namespaces y físicamente en shards para escalar.

ANN Engine y Storage

Un motor de búsqueda aproximada (HNSW) encuentra vecinos rápido; el storage distribuido persiste todo.

Escalabilidad

Sharding

Particiona los vectores en múltiples nodos para distribuir la carga a medida que crece el índice.

Réplicas

Copias de cada shard que dan alta disponibilidad y failover automático si un nodo falla.

El resultado

Escala horizontal + replicación = rendimiento, disponibilidad y tolerancia a fallos.

Cloud vs. Local

Pinecone (Cloud)

Servicio gestionado, escalabilidad elástica y multi-tenant nativo con namespaces.

ChromaDB / FAISS

Locales y gratuitos, ideales para prototipar, pero limitados en escala y sin replicación.

¿Cuándo migrar?

Cuando necesitas alta disponibilidad, millones de vectores, o servir a múltiples clientes.

Momentos de la Clase:

Actividad colaborativa

Para pensar

Para recordar

Coder Tips

Práctica - Unidad 1

Tu infraestructura serverless

Objetivo: configurar tu primera infraestructura vectorial serverless en la nube.

Manos a la obra:

- Inicializá el cliente de Pinecone y creá un índice con ServerlessSpec (cloud='aws', region='us-east-1').
- Hacé un upsert de prueba usando un namespace, y consultá describe_index_stats().

Resultado: tu primer índice en la nube, listo para recibir datos reales.

02

Ingesta masiva y metadatos

Batch Upsert

El corazón de la escalabilidad

Insertar vectores uno por uno es ineficiente: el overhead de red y los límites de la API frenan todo.

La estrategia correcta es el Batch Upsert: agrupar vectores y subirlos juntos.

Tamaño ideal: batches de entre 100 y 200 vectores, el punto dulce entre velocidad y estabilidad.

Puro vs. Filtro

Búsqueda semántica pura

Buscar "factura de Amazon" entre 10 millones de documentos puede traer "facturas de Azure": son semánticamente similares, pero no son lo que buscabas.

Con metadatos (Hard Filter)

Filtrás por user_id, año o categoría antes o durante la búsqueda semántica, acotando drásticamente el ruido.

El resultado

Menor espacio de búsqueda, menor latencia y resultados mucho más relevantes para el usuario.

El Riesgo del Schema Drift

Un solo esquema de metadatos para todos

Un error común es no estandarizar los metadatos entre distintos procesos de ingesta.

Mal: un proceso guarda la fecha como `ingest_date` y otro como `date_created`. Tus filtros van a fallar silenciosamente.

Bien: definir una interfaz de metadatos estricta y compartida por todos los pipelines de ingesta.

Casos de Uso de Metadatos

Filtros que hacen la diferencia

Los metadatos habilitan consultas SQL-like sobre tus vectores. Estos son los casos de uso más comunes.

- **Multi-tenancy:** filtrar por `user_id` u `org_id` para que un usuario nunca vea datos de otro.
- **Temporalidad:** filtrar por `year > 2023` para evitar información obsoleta.
- **Categorización:** limitar el contexto a `department == 'legal'`, por ejemplo.

Filtro en Pinecone

Vector + metadatos, en una sola consulta

Una consulta de Pinecone combina el vector de búsqueda con un filtro estructurado de metadatos.

top_k=10: cuántos resultados querés recuperar como máximo.

- **filter category \$eq "legal":** solo compara contra documentos de esa categoría.

filter status \$in ["reviewed", "final"]: además, solo documentos ya revisados o finales.

Document Loaders

La puerta de entrada a tus datos

Los Document Loaders se conectan a la fuente (archivo, URL o API), leen el contenido y lo convierten en un Document Object.

Ese objeto separa dos cosas: `page_content` (el texto extraído) y metadata (fuente, autor, página, fecha).

Esta separación es la base de todo lo que viene después: limpieza, chunking y filtrado por metadatos.

Limpieza y Normalización

Eliminar Ruido

Encabezados, pies de página repetidos y caracteres de una codificación rota.

Normalizar

Unicode, acentos, comillas y espacios o saltos de línea múltiples, a un formato consistente.

Deduplicar

Eliminar texto repetido dentro de un documento y entre documentos distintos.

Enriquecimiento

Metadata que mejora la búsqueda

Detección de idioma y clasificación temática: identifican de qué trata cada documento automáticamente.

Extracción de entidades y palabras clave: personas, organizaciones, fechas y términos relevantes.

Un resumen automático y esta metadata enriquecida mejoran directamente la precisión de la recuperación híbrida.

Chunking Profesional Avanzado

Semantic Chunking

Agrupar por similitud semántica: alta conservación de contexto, aunque con mayor costo computacional.

Parent Document

Indexa chunks pequeños pero recupera el documento padre completo, para respuestas con más contexto.

Markdown / Code Aware

Respetar la estructura del formato: títulos, bloques de código o funciones, sin cortarlos por la mitad.

Metadata Estratégica

Un esquema, no un campo suelto

Campos típicos: source, author, language, category, department, tags, doc_type y version.

Definir este esquema desde el día uno evita el Schema Drift que ya vimos en la Unidad 2.

Bien diseñada, la metadata permite filtros eficientes combinados con la búsqueda vectorial.

Multi-Tenant

Namespace por Cliente

Cada cliente o equipo tiene su propio espacio lógico dentro del mismo índice.

Aislamiento y Seguridad

Un cliente nunca accede a los datos de otro, aunque compartan la misma infraestructura.

El resultado

Reduce costos centralizando infraestructura, sin sacrificar el aislamiento de los datos.

Ciclo de Vida del Vector

De la ingesta a la reindexación

Embedding: se genera el vector del contenido nuevo o modificado.

Upsert: se inserta o actualiza ese vector en Pinecone, junto con su metadata.

Consulta: se busca por similitud, filtrando opcionalmente por metadata.

Actualización y Delete: se reindexa solo cuando realmente es necesario, no todo el índice de una vez.

Momentos de la Clase:

Para recordar

Coder Tips

Material ampliado

Cuestionario de tarea

#FindTheBug

La palabra clave del día es Schema Drift: si dos procesos nombran un mismo campo distinto, tus filtros fallan sin avisar. Documentá tu esquema desde el día uno.

Práctica - Unidad 2

Tu pipeline de ingesta masiva

Objetivo: construir un pipeline de ingesta que procese documentos en batches y los filtre por metadatos.

Manos a la obra:

- Implementá `create_metadata()` incluyendo categoría, autor, fecha y tamaño del texto.
- Implementá `process_and_upsert_batches()` dividiendo la lista de documentos en lotes.

Resultado: tu `IngestionPipeline` listo para ingesta masiva y búsqueda filtrada por categoría.

Break,

andá por un cafecito
o mate, ¡y volvemos!



03

Métricas y recuperación híbrida

Precision, Recall y F1

Las métricas que importan en RAG

Precision: cuánto de lo que el sistema recuperó es realmente relevante. Evita entregarle ruido al LLM.

Recall: qué proporción de todos los documentos relevantes existentes fue efectivamente encontrada.

Recall@k: si al menos un documento relevante aparece entre los primeros k resultados recuperados.

F1-Score: la media armónica entre precisión y recall, un único número que resume el rendimiento.

BM25 vs. Vector

BM25

Se basa en coincidencias exactas de términos: frecuencia y rareza de palabras. Excelente para nombres propios, códigos de error o términos técnicos.

Embeddings

Capturan el significado conceptual y la intención del usuario, manejando sinónimos y variaciones que BM25 ignoraría.

Búsqueda híbrida

Combina ambos para capturar lo mejor de los dos mundos: precisión léxica y comprensión semántica.

RRF y Re-ranking

Fusionar y refinar resultados

RRF (Reciprocal Rank Fusion) combina rankings de BM25 y vectores según la posición de cada documento, no su puntaje absoluto.

Es robusto porque no necesita normalizar puntuaciones que vienen de escalas totalmente distintas.

Re-ranking: una segunda etapa donde un Cross-Encoder evalúa con más precisión el top-k inicial.

El Cross-Encoder procesa la consulta y el documento juntos: muy preciso, pero demasiado lento para buscar en millones de documentos.

Calidad: Dónde se Gana

No depende solo del LLM

El modelo de embeddings pesa el 25% del impacto en la calidad final; el chunking, un 20%.

El retriever, el top-k y el re-ranking suman otro 35% combinado.

El prompt y el LLM, ya al final de la cadena, explican apenas un 10%: la calidad se construye mucho antes.

Retrieval

1. Consulta: el usuario ingresa su pregunta.
2. Embedding: se convierte la consulta en un vector semántico.
3. ANN Search: búsqueda aproximada en el índice vectorial.
4. Top-K inicial: se obtienen los documentos más similares.
5. Re-ranking: un cross-encoder reordena por precisión real.
6. Context Compression + Prompt Builder: se arma el prompt final para el LLM.

Bi vs. Cross

Bi-Encoder

Vectoriza consulta y documento por separado:
rápido, escalable y de bajo costo.

Cross-Encoder

Procesa ambos juntos: mucha más precisión, pero
mayor latencia y costo computacional.

En la práctica

Bi-Encoder para el retrieval inicial; Cross-Encoder solo para re-ranear el top-k final.

Técnicas Modernas de Recuperación

Hybrid Search & MMR

Combina BM25 con embeddings; MMR selecciona resultados diversos evitando redundancia.

Multi-Query & HyDE

Genera variantes de la consulta, o una respuesta hipotética, para buscar con más cobertura.

Parent & Self-Query

Recupera el documento padre completo, o construye filtros automáticos desde el lenguaje natural.

Lost in the Middle

La atención no es uniforme

Los modelos prestan más atención al inicio y al final de un contexto largo, perdiendo lo que está en el medio.

Mitigación: reordená el contexto poniendo lo más importante al inicio y al final.

También ayuda el Context Compression, y dividir en múltiples prompts secuenciales si el contexto es muy largo.

Elegir el Top-K

Top-K bajo (3-5)

Mayor precisión y menor costo, pero puede perder contexto relevante.

Top-K alto (10-20)

Mayor recall, pero más ruido, más costo y más latencia en cada consulta.

Regla práctica

Empezá con Top-5 o Top-10, y ajustá según los resultados de tu propia evaluación.

Métricas de Evaluación del Retrieval

Recall@K y Precision@K

Qué proporción de lo relevante se recuperó, y qué tan limpio viene el resultado.

MRR y NDCG

Qué tan arriba aparece el primer resultado relevante, y qué tan bien ordenado está el ranking.

Faithfulness y Relevancy

Si la respuesta es fiel al contexto entregado, y si responde lo que realmente se preguntó.

Optimización

Diagnóstico antes que parche

¿Falta contexto? Aumentá el overlap o probá otra estrategia de chunking.

¿Demasiados documentos? Bajá el Top-K.

¿Baja precisión? Aplicá re-ranking o metadata filters.

¿Hay alucinaciones? Mejorá la calidad del contexto y del retrieval, no solo el prompt.



Práctica - Unidad 4

Evaluando tu recuperador

Objetivo: evaluar cuantitativamente tu sistema de recuperación con un Golden Set de prueba.

Manos a la obra:

- Armá un pequeño set de preguntas con su documento_id_esperado conocido de antemano.
- Calculá Precision@5 y Recall@5 sobre los resultados recuperados para cada pregunta.

Cuidado: nunca sumes directamente scores de BM25 y de similitud vectorial, sus escalas son distintas.

PE

Pre-Entrega 4

¿Qué debes construir?

Pre-Entrega 4

Debés entregar un Módulo de Recuperación Escalable, corriendo en la nube con Pinecone.

- **Pipeline de Ingesta:** procesa documentos y los sube a Pinecone Serverless con metadatos avanzados.
- **Hybrid Retriever:** combina búsqueda vectorial con búsqueda léxica (BM25) para mayor precisión.

Script de Evaluación: calcula Precision@k y Recall@k sobre un Golden Set de preguntas y respuestas.

Tu misión

El entregable

 **Entrega: repositorio de GitHub con el pipeline de ingesta, el recuperador híbrido y el script de evaluación**

 **El README debe incluir instrucciones para replicar el índice de Pinecone**

Construí un sistema RAG que escale a la nube y que puedas demostrar que efectivamente recupera bien, no solo que "parece funcionar".

Pasos

1. Creá un índice Serverless en Pinecone (dimensión 1536 si usás `text-embedding-3-small`).
2. Guardá el texto original dentro de los metadatos, para evitar consultas extra a otra base de datos.
3. Configurá el `PineconeVectorStore` de LangChain o el SDK nativo para tu motor de búsqueda.
4. Sumá un `BM25Retriever` y combinalo con Pinecone usando un `EnsembleRetriever`.
5. Armá un benchmark de preguntas y medí cuántos documentos esperados aparecen en el Top-5.

Errores a Evitar

Antes de entregar, revisá esto

Mismatch de Dimensiones: subir embeddings de 1536 a un índice configurado con 512 o 768.

- Ignorar el Namespace: en apps multi-inquilino, no usarlos vuelve la búsqueda ruidosa y lenta.
- Subestimar el Chunking: chunks muy chicos pierden contexto; muy grandes diluyen precisión. Buscá 500–800 tokens.
- Sumar scores heterogéneos: BM25 y similitud coseno no comparten escala; usá RRF, no una suma directa.

¿Cómo se evalúa?

· 100 pts

Infraestructura Cloud · 15%

Correcta inicialización del índice serverless y manejo seguro de variables de entorno.

Ingesta y Metadatos · 25%

Procesar documentos técnicos, fragmentarlos y persistirlos con metadatos útiles.

Recuperador Híbrido y Evaluación · 60%

Diseño de RAGSystem con EnsembleRetriever, y script evaluate.py para Recall@5 y Precision@5.

Síntesis del módulo

De local a escalable

Pasamos de ChromaDB local a una infraestructura vectorial en la nube que escala a millones de documentos.

- **El Batch Upsert y los metadatos bien diseñados son la diferencia entre un prototipo y un sistema de producción.**
- **La recuperación híbrida (BM25 + embeddings) y el re-ranking maximizan la precisión que le llega al LLM.**

Próximo módulo: de recuperar información pasamos a que el sistema razone y actúe por sí mismo.

Arquitecturas de producción a escala

Sesión en Vivo 4

En vivo resolveremos un caso de 'Recuperación Fallida': documentos semánticamente similares pero contextualmente irrelevantes.

- **Diseñaremos en grupos una arquitectura para un cliente de LegalTech con 10M de contratos y latencia menor a 200ms.**
- **Compararemos Namespaces vs. Metadatos: aislar datos vs. filtrar dentro de un mismo segmento.**

Traé tu acceso a la consola de Pinecone y tu Pre-Entrega 4 como base para los ejercicios.

¿Preguntas?

De cientos a millones de documentos, sin perder precisión. Soltá tus dudas del módulo.



Glosario

1. Pinecone: base de datos vectorial nativa de la nube, con arquitectura serverless.
2. Index: contenedor principal que define dimensión de vectores y métrica de similitud.
3. Namespace: partición lógica dentro de un índice, usada para multi-tenancy o entornos.
4. Serverless: arquitectura donde no gestionás nodos; el almacenamiento escala automáticamente.
5. Batch Upsert: insertar o actualizar vectores en lotes, en vez de uno por uno.
6. Hard Filter: filtro rígido por metadatos aplicado antes o durante la búsqueda semántica.

1. Schema Drift: inconsistencia entre procesos al nombrar los mismos campos de metadatos.
2. Multi-tenancy: aislar los datos de distintos clientes o usuarios dentro del mismo sistema.
3. Precision: proporción de resultados recuperados que son realmente relevantes.
4. Recall: proporción de todos los documentos relevantes que el sistema logró encontrar.
5. Recall@k / Precision@k: precisión o recall medidos sobre los primeros k resultados.
6. F1-Score: media armónica entre precisión y recall.

1. BM25: algoritmo de búsqueda léxica basado en frecuencia y rareza de términos.
2. RRF (Reciprocal Rank Fusion): fusiona rankings de distintas fuentes según su posición, no su score.
3. Re-ranking: segunda etapa que reordena el top-k inicial con un modelo más preciso.
4. Cross-Encoder: modelo que evalúa la consulta y el documento juntos; preciso pero lento.
5. Golden Set: conjunto de preguntas con respuesta conocida, usado para evaluar un sistema RAG.
6. EnsembleRetriever: combina múltiples recuperadores (por ejemplo BM25 y vectorial) en uno solo.

1. FAISS: librería de Meta para búsqueda vectorial local, en memoria, sin persistencia nativa.
2. LanceDB: base de datos vectorial local con almacenamiento columnar.
3. Weaviate / Milvus: bases de datos vectoriales open-source para despliegues cloud u on-premise.
4. ANN (Approximate Nearest Neighbor): búsqueda aproximada del vecino más cercano, mucho más rápida que la exacta.
5. HNSW: algoritmo de grafos usado por motores ANN para búsquedas de alta velocidad.
6. Sharding: particionar un índice en múltiples nodos para distribuir carga.

1. Réplica: copia de un shard usada para alta disponibilidad y failover automático.
2. Document Loader: componente que lee una fuente y la convierte en un Document Object.
3. page_content: el texto extraído de un documento, dentro de su Document Object.
4. Bi-Encoder: modelo que vectoriza consulta y documento por separado; rápido y escalable.
5. MMR (Maximal Marginal Relevance): prioriza resultados relevantes pero diversos entre sí.
6. Parent Document Retriever: recupera el documento padre completo a partir de un chunk pequeño.

1. Contextual Compression: reduce el contexto recuperado, conservando solo lo más relevante.
2. Self-Query Retriever: construye filtros de metadata desde la consulta en lenguaje natural.
3. Multi-Query Retriever: genera variantes de una consulta para ampliar la cobertura de búsqueda.
4. HyDE: genera una respuesta hipotética a la consulta y busca documentos similares a ella.
5. NDCG: métrica que evalúa qué tan bien ordenado está un ranking de resultados.
6. MRR (Mean Reciprocal Rank): mide qué tan arriba aparece el primer resultado relevante.

1. Hit Rate: proporción de consultas donde se recuperó al menos un resultado relevante.
2. Context Precision / Recall: qué tan limpio y qué tan completo es el contexto entregado al LLM.
3. Faithfulness: qué tan fiel es la respuesta del LLM al contexto que se le entregó.
4. Answer Relevancy: qué tan relevante es la respuesta frente a lo que realmente se preguntó.
5. Lost in the Middle: pérdida de atención del modelo sobre info en el medio de un contexto largo.
6. Top-K: cantidad de resultados más similares que se recuperan en una consulta.

¡Gracias!